14. EXPLORING PRO-CODE EXTENSION OF A ZOHO CREATOR APPLICATION USING ZOHO CATALYST / CUSTOM FUNCTIONS

Aim

To extend the Insurance Claim Management System in Zoho Creator using custom Deluge functions and, where required, Zoho Catalyst serverless services for advanced computation, external API integration, and intensive processing.

Theory

Zoho Creator provides forms, reports, workflows, and Deluge scripting for rapid application development. More complex insurance claim requirements may benefit from reusable custom functions or pro-code backend services.

1. Custom Deluge Functions

Custom Deluge functions encapsulate reusable business logic inside Zoho Creator. In the Insurance Claim Management System, they may be used to calculate estimated payouts, validate claim information, apply coverage rules, or call external REST APIs. For example, a function can calculate an estimated payout from the approved claim amount and coverage percentage.

2. Zoho Catalyst Serverless Pro-Code Integration

Zoho Catalyst supports serverless backend development using languages such as Java, Node.js, and Python. If the claim application requires processing that is unsuitable for a simple Creator workflow—such as intensive document processing, specialized fraud-analysis services, or complex external integrations—the work can be handled by a Catalyst function and called through an authenticated API.

Procedure

Step 1: Create a Custom Deluge Function

1. Open the Insurance Claim Management System in Zoho Creator.
2. Navigate to the application logic/functions area available in your Creator interface.
3. Choose **Create Function**, then define a suitable name, input arguments, and return type. For example, a payout function can accept an approved claim amount and coverage percentage.
4. Write and test the Deluge logic, then save the function.
5. Associate the function with the relevant claim form workflow, such as after a claim is submitted or approved.

Example Deluge logic — estimated payout

```
estimatedPayout = input.Claim_Amount * input.Coverage_Percentage / 100;
```

Use the actual field link names from your application. If the function is called from a workflow, pass or reference the relevant field values according to the function configuration.

Step 2: Set Up a Zoho Catalyst Project (Optional)

1. Open the Zoho Catalyst Console and create a project if you need a separate pro-code backend.
2. Use the supported development environment or CLI to create a serverless function in Node.js, Java, or Python.
3. Implement the advanced task required by the insurance application, such as processing a document or calling a specialist analysis service.
4. Deploy the function and note its API endpoint and authentication requirements.

Step 3: Connect Creator to Catalyst Using an API

1. In Creator, open the Connections area under Microservices/integration settings and create the required authenticated connection if your account supports it.
2. Configure the Catalyst endpoint and authentication method securely. Do not expose API keys or tokens in a public form or hardcode sensitive credentials in shared scripts.
3. Use an `invokeurl` request in a custom function to send only the required claim data to the endpoint.
4. Parse the response and use the returned result in the appropriate claim field or report.

Example request structure

```
response = invokeurl
[
  url : "YOUR_AUTHENTICATED_CATALYST_ENDPOINT"
  type : POST
  parameters : {"claimId": input.ID, "claimAmount": input.Claim_Amount}
  connection : "your_connection_link_name"
];
```

Step 4: Test and Validate the Workflow

1. Submit a test claim or use a safe test record.
2. Confirm that the Creator workflow triggers the custom function.
3. If Catalyst is used, verify that the request reaches the endpoint and returns the expected response.
4. Check the estimated payout or other result in the claim record and report.
5. Test invalid inputs and API failures so the application handles errors safely.

Result

The Insurance Claim Management System can be extended with reusable Deluge functions for claim validation and payout calculations. When advanced processing is required, Zoho Catalyst can provide a separate pro-code backend through authenticated API calls. Actual implementation should be tested with the field link names, connections, and features available in the account.